National University of Computer and Emerging Sciences

Karachi Campus

Operating Systems (CS2006)

Mid-01 Exam

Date: September 19th 2026

Total Time (Hrs): 1

Total Marks: 15

Total Questions: 2

Course Instructor(s)

Mr. Sandesh Kumar

_____ _____ _____

Roll No Section Student Signature

Do not write below this line

Attempt all the questions and do not use markers or pencils to write your answers

CLO#1: Describe, discuss and analyze services provided by modern operating systems

Q1. [1.5 marks x 5 = 7.5 marks, Estimated Time: 25 Minutes]

Write short textual answers

Note: No Drawings. Explain all technical terms in your answer to get full marks.

a) A hospital needs a system that runs scheduled batch reports on patient records and must respond instantly to a ventilator alarm. Identify the most suitable computing environment/OS type for this scenario and justify your choice, ruling out the alternatives.

Real-Time Operating System (RTOS), because it can guarantee a bounded, predictable response time to the alarm event, which is essential since a delayed response could cause patient harm.

Justification: A batch-processing system executes submitted jobs in sequence (or by priority queue) with no guaranteed deadline, it optimizes for throughput, not response time. This makes batch processing unsafe and unsuitable here. An RTOS, in contrast, is designed so that critical tasks always meet their deadline, while lower-priority background tasks run whenever CPU time is free

Marking Criterion	**Marks**
Correctly identifies RTOS (Real-Time Operating System) as the OS type	0.5
Justifies the choice using a technical property of RTOS (bounded/guaranteed response time, deadline-driven scheduling)	0.5
Explicitly rules out Batch Processing with a technical reason (no deadline guarantee / throughput-oriented / unsafe delay)	0.5
Total	**1.5**

b) What is the role of the interrupt vector table? Trace the steps taken by hardware and OS from the moment a device raises an interrupt to the moment the interrupted process resumes

The Interrupt Vector Table is maintained by the OS that maps each interrupt number (IRQ line) to the memory address of its corresponding Interrupt Service Routine (ISR). It allows the CPU to jump directly to the correct handler for a given interrupt without searching, the interrupt number is used as an index into the table.

Steps from interrupt to resumption:

1. A device raises an interrupt signal on its IRQ line.
2. The CPU finishes its current instruction cycle and checks for pending interrupts (or the interrupt controller signals the CPU).

3. The CPU saves the interrupted process's context (program counter, registers, status word) typically onto the stack / into its PCB.
4. The CPU uses the interrupt number to index into the Interrupt Vector Table and retrieves the address of the matching ISR.
5. Control transfers to the ISR, which runs in kernel mode and services the device (e.g., transfers data, acknowledges/clears the interrupt).
6. On completion of the ISR, the OS restores the saved context of the process that was interrupted.
7. The interrupted process resumes execution from exactly where it left off (or, if appropriate, the scheduler picks a different ready process to run next).

Marking Criterion	**Marks**
Correct definition of the IVT (maps interrupt number → ISR address)	0.5
Correct ordering of hardware steps (signal → context save → vector lookup)	0.5
Correct completion steps (ISR execution, context restore, resumption)	0.5
Total	**1.5**

c) Compare single-processor, multiprocessor (SMP), and clustered systems. For a system requiring high availability with minimal downtime during hardware failure, which architecture is most suitable and why?

Single-processor: one CPU executes one instruction stream at a time.

Multiprocessor (SMP): multiple CPUs share the same memory and run under a single OS instance.

Clustered systems: two or more complete, independent computer systems (each with its own OS instance and often its own storage) connected via a fast interconnect, working cooperatively and monitoring each other.

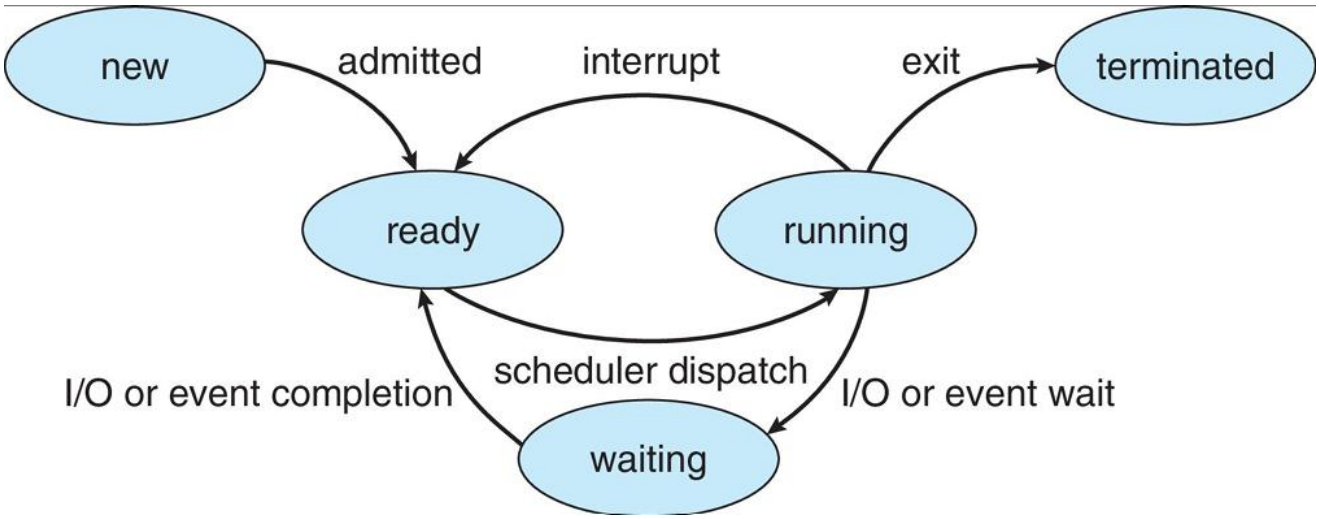
For high availability with minimal downtime, a clustered system is most suitable. If one node fails, another independent node takes over (failover) almost immediately, because the nodes do not share a single OS instance or a single point of hardware failure.

Marking Criterion	**Marks**
Correct distinguishing feature for each of the three architectures	0.75
Correct final choice (clustered system) with a technical justification tied to failover/independent nodes	0.5
Explicitly contrasts against SMP and single-processor's single points of failure	0.25
Total	**1.5**

Give only labelled diagrams (DO NOT use pencil)

Note: No grading of textual answers here.

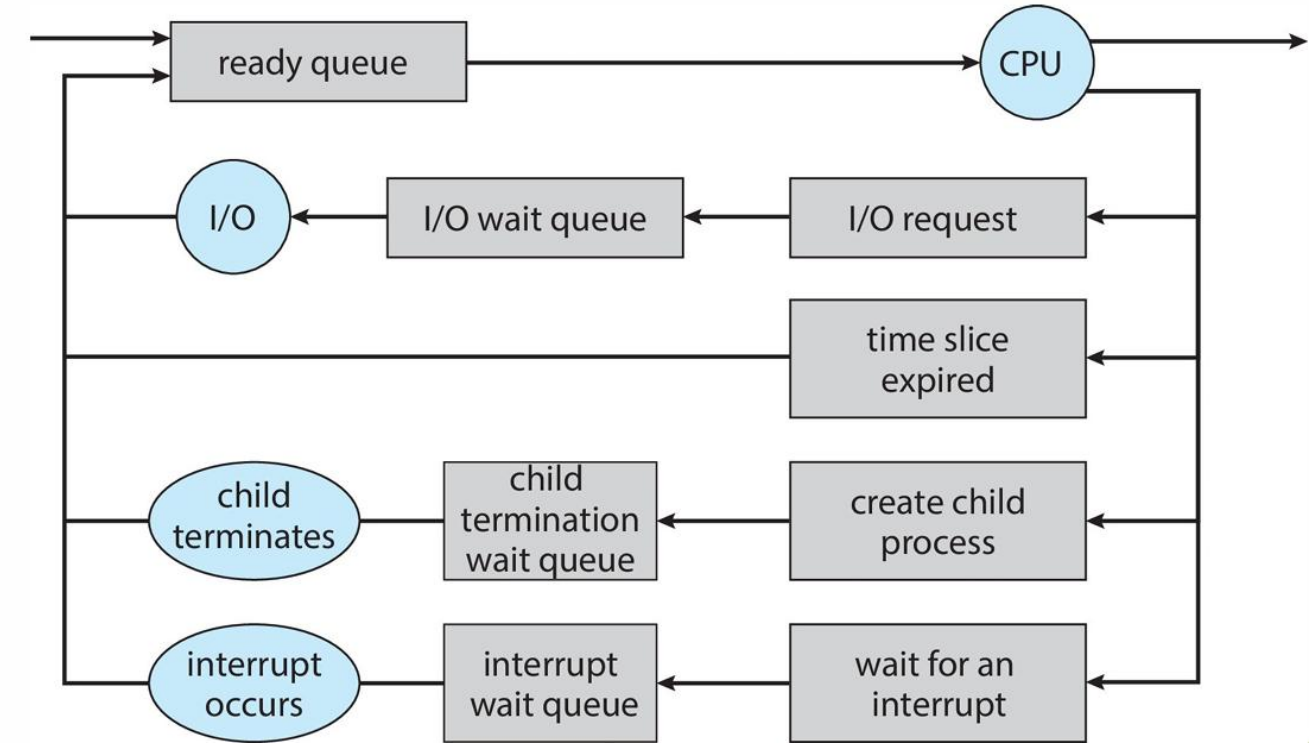
d) Draw and label the process state diagram and explain the transition that occurs when a process issues a `read()` system call on a slow device



when a Running process issues read() on a slow device, the OS moves it from Running → Waiting (not Running → Ready), because the requested data is not yet available. The OS saves the process's context into its PCB, places the process into the device's wait queue, and dispatches another ready process onto the CPU. When the I/O completes (device interrupt), the process moves Waiting → Ready, and later gets dispatched back to Running.

Marking Criterion	**Marks**
All five states present and correctly labelled (New, Ready, Running, Waiting, Terminated)	0.5
All required transition arrows present and correctly directed	0.5
Running → Waiting transition explicitly and correctly labelled with the read()/I-O-wait trigger (the key transition being tested)	0.5
Total	**1.5**

e) Illustrate four or more different ways a process can wait in different queues after leaving the CPU, till it is admitted back into the ready queue



Marking Criterion	Marks
At least four distinct, valid wait queues shown (0.3 marks each, up to 1.2)	1.2
Correct labelling of the trigger that returns each process to the Ready queue (e.g., I/O complete, timer expires, child exits)	0.3
Total	**1.5**

CLO#2: Understand, design, and implement solution employing concepts of Processes/Threads

Q2: [4 marks + 3.5 marks = 7.5 marks, Estimated Time: 35 Minutes]

Code Implementation

a) Write a C program where a parent process reads a string from the user and sends it to two child processes using separate pipes: Child 1 removes all vowels from the string; Child 2 counts the total number of words. Both children return results to the parent via their own pipes, and the parent prints both. Use `fork(), pipe(), read(), and write()`.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <ctype.h>
#include <sys/wait.h>

#define MAX 200

int main(void) {
    char input[MAX];
    printf("Enter a string: ");
    if (fgets(input, MAX, stdin) == NULL) {
        fprintf(stderr, "Failed to read input\n");
```

```
    exit(1);
}
input[strcspn(input, "\n")] = '\0';   // strip trailing newline

int pipe1[2], pipe2[2];            // parent -> children
int pipe1_back[2], pipe2_back[2];  // children -> parent

if (pipe(pipe1) == -1 || pipe(pipe2) == -1 ||
    pipe(pipe1_back) == -1 || pipe(pipe2_back) == -1) {
    perror("pipe"); exit(1);
}

pid_t c1 = fork();
if (c1 == -1) { perror("fork"); exit(1); }

if (c1 == 0) {                       /* Child 1: remove vowels */
    close(pipe1[1]); close(pipe1_back[0]);
    close(pipe2[0]); close(pipe2[1]);
    close(pipe2_back[0]); close(pipe2_back[1]);

    char s[MAX], out[MAX];
    int n = read(pipe1[0], s, MAX);
    if (n < 0) { perror("read"); exit(1); }
    s[n] = '\0';
    close(pipe1[0]);

    int j = 0;
    for (int i = 0; s[i]; i++) {
        char c = tolower(s[i]);
        if (c!='a' && c!='e' && c!='i' && c!='o' && c!='u')
            out[j++] = s[i];
    }
    out[j] = '\0';

    if (write(pipe1_back[1], out, strlen(out)+1) == -1) perror("write");
    close(pipe1_back[1]);
    exit(0);
}

pid_t c2 = fork();
if (c2 == -1) { perror("fork"); exit(1); }

if (c2 == 0) {                       /* Child 2: count words */
    close(pipe2[1]); close(pipe2_back[0]);
    close(pipe1[0]); close(pipe1[1]);
    close(pipe1_back[0]); close(pipe1_back[1]);

    char s[MAX];
    int n = read(pipe2[0], s, MAX);
    if (n < 0) { perror("read"); exit(1); }
    s[n] = '\0';
    close(pipe2[0]);

    int count = 0, in_word = 0;
    for (int i = 0; s[i]; i++) {
        if (!isspace((unsigned char)s[i])) {
            if (!in_word) { count++; in_word = 1; }
        } else in_word = 0;
    }
```

```
        char result[32];
        snprintf(result, sizeof(result), "%d", count);
        if (write(pipe2_back[1], result, strlen(result)+1) == -1) perror("write");
        close(pipe2_back[1]);
        exit(0);
    }

    /* Parent */
    close(pipe1[0]); close(pipe2[0]);
    close(pipe1_back[1]); close(pipe2_back[1]);

    if (write(pipe1[1], input, strlen(input)+1) == -1) perror("write");
    if (write(pipe2[1], input, strlen(input)+1) == -1) perror("write");
    close(pipe1[1]); close(pipe2[1]);

    char vowel_removed[MAX], word_count[32];
    int n1 = read(pipe1_back[0], vowel_removed, MAX);
    if (n1 >= 0) vowel_removed[n1] = '\0';
    int n2 = read(pipe2_back[0], word_count, 32);
    if (n2 >= 0) word_count[n2] = '\0';
    close(pipe1_back[0]); close(pipe2_back[0]);

    wait(NULL); wait(NULL);

    printf("Child 1 (vowels removed): %s\n", vowel_removed);
    printf("Child 2 (word count): %s\n", word_count);
    return 0;
}
```

Marking Criterion	Marks
Correct pipe() creation (4 pipes, correctly split by direction) and fork() logic (2 forks, correct parent/child branching)	1.25
Child 1 correctly removes vowels and sends result back on its own pipe	0.75
Child 2 correctly counts words and sends result back on its own pipe	0.75
Unused pipe ends closed appropriately in each process (parent and both children)	0.5
Error checking on fork()/pipe()/read()/write() (or equivalent), and parent correctly reads and prints both results	0.75
Total	**4**

Understanding code semantics

b) Suppose a process executes the code shown below.

```
int main() {
    int y = 2;
    if (fork() != 0) {          /* Fork A */
        y = y + 3;
        fork();                 /* Fork B */
    }
    y = y * 2;
    fork();                     /* Fork C */
    y++;
    if (y > 6)
        fork();                 /* Fork D */
    printf("y=%d, PID=%d, PPID=%d\n", y, getpid(), getppid());
    return 0;
}
```

Dry-run the execution, showing the value of y computed by each resultant process. Draw a labelled parent–child relationship diagram with descriptive hints (which fork created which process, and what value of y each one holds). Identify the total number of processes created, and state which processes are siblings and which are grandchildren of the original process. [1.5 + 1.5 + 0.5 = 3.5 marks]

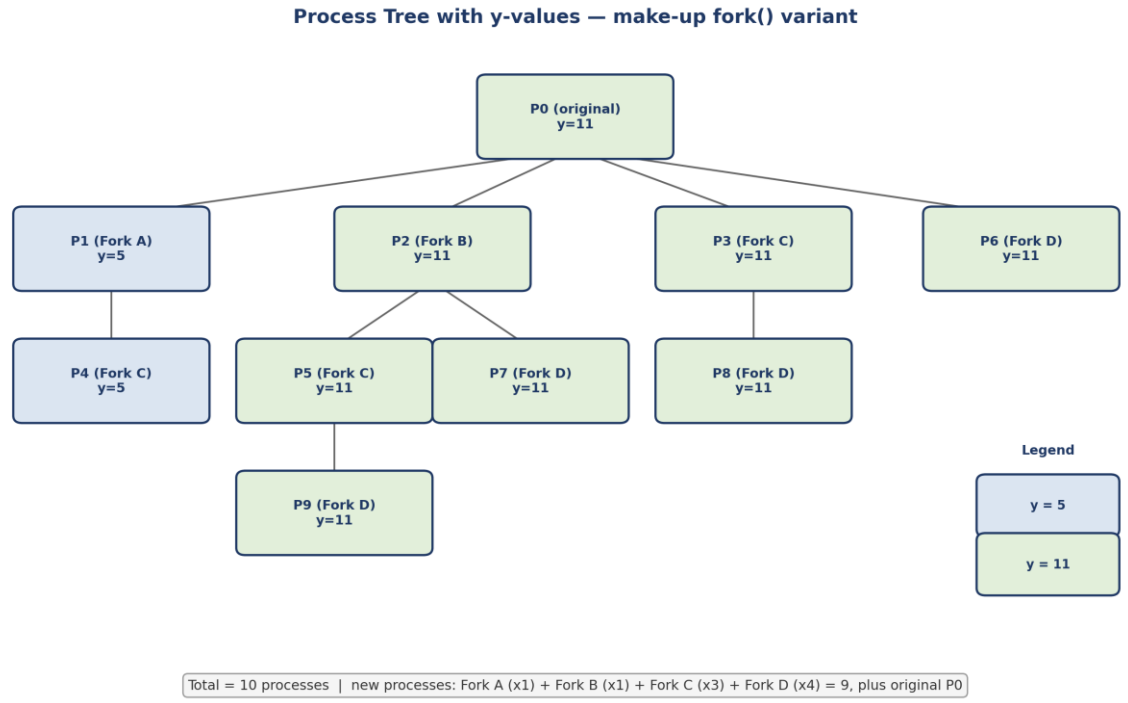
Dry-Run Steps:

1. Start: only P0 exists, y = 2.
2. Fork A, if (fork() != 0): P0 forks P1. In P0 fork() returns the child's PID (nonzero), so the condition is true and P0 enters the block. In P1 fork() returns 0, so the condition is false and P1 skips the block (y stays 2).
3. Inside the block (P0 only): y = 2 + 3 = 5. Fork B is executed by P0 only and creates P2, which copies y = 5.
4. y = y * 2 is executed by all three live processes: P0 = 10, P2 = 10, P1 = 4.
5. Fork C is executed by all three: P0 creates P3 (y = 10), P1 creates P4 (y = 4), P2 creates P5 (y = 10).
6. y++ is executed by all six (children continue after the fork line too): P0 = 11, P2 = 11, P3 = 11, P5 = 11, P1 = 5, P4 = 5.
7. if (y > 6): true for P0, P2, P3, P5 (y = 11) and false for P1, P4 (y = 5). Fork D therefore runs in four processes: P0 creates P6, P2 creates P7, P3 creates P8, P5 creates P9, each with y = 11.
8. All 10 processes then execute printf.

Process	Created by	Parent	y at printf
P0	original	—	11
P1	Fork A	P0	5
P2	Fork B	P0	11
P3	Fork C	P0	11
P4	Fork C	P1	5
P5	Fork C	P2	11
P6	Fork D	P0	11
P7	Fork D	P2	11
P8	Fork D	P3	11
P9	Fork D	P5	11

Printed values: y = 5 for P1 and P4; y = 11 for the other eight processes.

Process Tree



Total Siblings and Grandchildren

Total processes = 10: the original P0 plus 9 new ones (Fork A creates 1, Fork B creates 1, Fork C creates 3, Fork D creates 4).

Siblings (same direct parent): {P1, P2, P3, P6} are children of P0; {P5, P7} are children of P2. P4, P8 and P9 are only children, so they have no siblings.

Grandchildren of P0: P4 (via P1), P5 and P7 (via P2), and P8 (via P3). P9 is a great-grandchild (P0 → P2 → P5 → P9).

Marking Criterion	**Marks**
Correct reasoning for the first condition (parent enters the block, child skips it) and correct y after Fork A/B (P0 = 5 in block, P1 stays 2)	0.5
y tracked correctly through y = y * 2, Fork C and y++ (values 11 and 5 reached correctly)	0.5
Fork D decided correctly by y > 6 (four processes fork, P1 and P4 do not) and final y values stated for all processes	0.5
Tree shows all 10 processes with correct parent–child links	0.75
Each node labelled with the fork that created it and its y value (descriptive hints)	0.75
Correct total (10) stated	0.25
At least one correct sibling set and one correct grandchild relationship	0.25
Total	**3.5**